additional tools) from a more straightforward implementation that works with lists of results.

Wand describes several program transformations, including the introduction of an accumulating parameter shown in Sect. 11.3.2, in terms of continuations [13].

In 1993, Reynolds [8] looks back on the many times continuations were discovered (or invented).

Guy Steele [10] shows that if functions (lambda abstractions) and function calls are implemented properly, via continuation-passing style, then all other constructs can be implemented efficiently in terms of these. This idea was realized by Steele in the 1978 Rabbit compiler for Scheme [11], the first compiler to use continuation-passing style, giving a breakthrough in efficiency of functional language implementations.

Andrew Appel [1] describes the design of the Standard ML of New Jersey (SML/NJ) compiler which initially transforms the entire program into continuation-passing style, as suggested by Steele.

The complete text of Griswold and Griswold's 1996 book on the Icon programming language is available for free [5].

11.11 Exercises

The main goal of these exercises is to master the somewhat mind-bending notion of continuation. But remember that a continuation is just something — usually a function — that represents the rest of a computation.

Exercise 11.1 (i) Write a continuation-passing (CPS) version `lenc : 'a list ->(int ->'b) ->'b` of the list length function `len`:

```
let rec len xs =
    match xs with
    | []    -> 0
    | x::xr -> 1 + len xr;;
```

Try calling the resulting function with `lenc [2; 5; 7] id`, where the initial continuation `let id = fun v -> v` is the identity function, and with `lenc [2; 5; 7] (printf "The answer is '% d' \n")`, where the initial continuation consumes the result and prints it.
(ii) What happens if you call it as `lenc xs (fun v -> 2*v)` instead?

(iii) Write also a tail-recursive version `leni : int list -> int -> int` of the length function, whose second parameter is an accumulating parameter. The function should be called as `leni xs 0`. What is the relation between `lenc` and `leni`?

Exercise 11.2 (i) Write a continuation-passing version `revc` of the list reversal function `rev`:

```
let rec rev xs =
    match xs with
    | []    -> []
    | x::xr -> rev xr @ [x];;
```

The resulting function `revc` should have type `'a list -> ('a list -> 'a list) -> 'a list` or a more general type such as `'a list -> ('a list -> 'b) -> 'b`. The function may be called as `revc xs id`, where `let id = fun v -> v` is the identity function.

(ii) What happens if you call it as `revc xs (fun v -> v @ v)` instead?

(iii) Write a tail-recursive reversal function `revi : 'a list -> 'a list-> 'a list`, whose second parameter is an accumulating parameter, and which should be called as `revi xs []`.

Exercise 11.3 Write a continuation-passing version `prodc : int list -> (int -> int) -> int` of the list product function `prod`:

```
let rec prod xs =
    match xs with
    | []    -> 1
    | x::xr -> x * prod xr;;
```

Exercise 11.4 Optimize the CPS version of the `prod` function above. It could terminate as soon as it encounters a zero in the list (because any list containing a zero will have product zero), assuming that its continuation simply multiplies the result by some factor. Try calling it in the same two ways as the `lenc` function in Exercise 11.1. Note that even if the non-tail-recursive `prod` were improved to return 0 when encountering a 0, the returned 0 would still be multiplied by all the `x` values previously encountered.

Write a tail-recursive version `prodi` of the `prod` function that also terminates as soon as it encounters a zero in the list.

Exercise 11.5 Write more examples using exceptions and exception handling in the small functional and imperative languages implemented in `Contfun.fs` and `Contimp.fs`, and run them using the given interpreters.

Exercise 11.6 What statements are in tail position in the simple imperative language implemented by `coExec1` in file `Contimp.fs`? Intuitively, the last statement in a statement block `{ ... }` is in tail position provided the entire block is. Can you argue that this is actually the case, looking at the interpreter `coExec1`?

Exercise 11.7 The `coExec1` version of the imperative language interpreter in file `Contimp.fs` supports a *statement* `Throw` to throw an exception. This `Throw`

statement is similar to `throw` in Java. Add an *expression* `EThrow` to the expression abstract syntax to permit throwing exceptions also inside an expression, as in F#'s `fail` expression. You will need to rewrite the expression interpreter `eval` in continuation-passing style; for instance, it must take a continuation as an additional argument. Consequently, you must also modify `coExec1` so that every call to `eval` has a continuation argument.

The return type of your new expression interpreter should be `answer` as for `coExec1`, and it should take a normal continuation of type (`int -> answer`) as argument, where `answer` is the exact same type used in the `coExec1` statement interpreter. (Like `coExec1`, your new expression interpreter need not take an error continuation, because we do not intend to implement exception handling.)

Your interpreter should be able to execute `run1 ex4` and `run1 ex5` where

```
let ex4 =
    Block[If(EThrow (Exn "Foo"), Block[], Block[])];;

let ex5 =
    While(EThrow (Exn "Foo"), Block[]);;
```

Exercise 11.8 The micro-Icon expression `2 * (1 to 4)` succeeds four times, with the values `2 4 6 8`. This can be shown by evaluating

```
open Icon;;
run (Every(Write(Prim("*", CstI 2, FromTo(1, 4)))));;
```

using the interpreter in `Icon.fs` and using abstract syntax instead of the concrete syntax `(2 * (1 to 4))`. We must use abstract syntax because we have not written lexer and parser specification for micro-Icon. A number of examples in abstract syntax are given at the end of the `Icon.fs` source file.

(i) Write an expression that produces and prints the values `3 5 7 9`. Write an expression that produces and prints the values `21 22 31 32 41 42`.

(ii) The micro-Icon language (like real Icon) has no boolean values. Instead, failure is used to mean `false`, and success means `true`. For instance, the less-than comparison operator (<) behaves as follows: `3 < 2` fails, and `3 < 4` succeeds (once) with the value 4. Similarly, thanks to backtracking, `3 < (1 to 5)` succeeds twice, giving the values 4 and 5. Use this to write an expression that prints the least multiple of 7 that is greater than 50.

(iii) Extend the abstract syntax with unary (one-argument) primitive functions, like this:

```
type expr =
  | ...
  | Prim1 of string * expr
```

Extend the interpreter `eval` to handle such unary primitives, and define two such primitives: (a) define a primitive `sqr` that computes the square $x \cdot x$ of its argument x; (b) define a primitive `even` that fails if its argument is odd, and succeeds if it is even (producing the argument as result). For instance, `square(3 to 6)` should succeed four times, with the results 9, 16, 25, 36, and `even(1 to 7)` should succeed three times with the results 2, 4, 6.

(iv) Define a unary primitive `multiples` that succeeds infinitely many times, producing all multiples of its argument. For instance, `multiples(3)` should produce 3, 6, 9, …. Note that `multiples(3 to 4)` would produce multiples of 3 forever, and would never backtrack to the subexpression `(3 to 4)` to begin producing multiples of 4.

Exercise 11.9 Write lexer and parser specifications for micro-Icon so Exercise 11.8 above could be solved using concrete syntax.

Exercise 11.10 (*For adventurous Java or C# hackers*): Implement a class-based abstract syntax and an interpreter for a backtracking Icon-style language subset, in the spirit of `Icon.fs`, but do it in C# or Java. If you use Java, you can draw some inspiration from method `facc` in `Factorial.java`. With C#, you should use lambda expressions (`v => v * 2`) or possibly the older-style anonymous delegates (`delegate(int v) { return v * 2; }`).

In any case, the result will probably appear rather incomprehensible, and could be used to impress people in the Friday bar. Nevertheless, it should not be too hard to write if you take a systematic approach.

Exercise 11.11 (*Project*) Implement a larger subset of the language Icon. This involves deciding on the subset, writing lexer and parser specifications, and writing an extended interpreter in the style of `Icon.fs`. The interpreter must at least handle assignable variables.

Exercise 11.12 (*Project*) Write a program to transform programs into continuation-passing style, using the Danvy and Filinski presentation [2] which distinguishes between administrative redexes and other redexes.

Exercise 11.13 (*Somewhat hairy project*) Extend a higher order functional language with the ability to capture the current (success) continuation, and to apply it. See papers by Danvy, Malmkjær, and Filinski. It would be a good idea to experiment with `call-with-current-continuation` in Scheme first.

References

1. Appel, A.W.: Compiling with Continuations. Cambridge University Press, Cambridge (1992)
2. Danvy, O., Filinski, A.: Representing control. A study of the CPS transformation. Math. Struct. Comput. Sci. **2**, 361–391 (1992)